Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине: “СПП”

Выполнил:
Студент 3 курса
Группы ПО-12
Мартынюк В. В.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета pytest

Задание 1: Написание тестов для мини-библиотеки покупок (shopping.py)

```
"""
Модуль shopping - мини-библиотека для управления корзиной покупок.

Этот модуль предоставляет класс Cart для управления корзиной товаров,
а также функции для логирования покупок и применения купонов.
"""

import requests


class Cart:
    """
    Класс корзины покупок.

    Позволяет добавлять товары, применять скидки и подсчитывать общую стоимость.
    """

    def __init__(self):
        """Инициализация пустой корзины."""
        self.items = []  # каждый элемент: (name, price)

    def add_item(self, name: str, price: float) -> None:
        """
        Добавление товара в корзину.

        Args:
            name: название товара
            price: цена товара

        Raises:
            ValueError: если цена отрицательная
        """
        if price < 0:
            raise ValueError("Price cannot be negative")
        self.items.append((name, price))

    def total(self) -> float:
        """
        Подсчёт общей стоимости всех товаров в корзине.
```

```
        Returns:
            Общая сумма всех товаров
        """
        return sum(price for _, price in self.items)

    def apply_discount(self, percent: float) -> None:
        """
        Применение скидки ко всем товарам в корзине.

        Args:
            percent: процент скидки (0-100)

        Raises:
            ValueError: если процент скидки вне диапазона [0, 100]
        """
        if percent < 0 or percent > 100:
            raise ValueError("Discount must be between 0 and 100")
        multiplier = (100 - percent) / 100.0
        self.items = [(name, price * multiplier) for name, price in self.items]

    def __len__(self) -> int:
        """Возвращает количество товаров в корзине."""
        return len(self.items)


def log_purchase(item: dict) -> None:
    """
    Логирование покупки в удалённую систему.

    Args:
        item: словарь с информацией о товаре

    Note:
        Добавлен timeout=30 для предотвращения зависания программы.
    """
    requests.post("https://example.com/log", json=item, timeout=30)


def apply_coupon(cart: Cart, coupon_code: str) -> None:
    """
    Применение купона к корзине.

    Args:
        cart: объект корзины
        coupon_code: код купона
```

```
    Raises:
        ValueError: если купон недействителен
    """
    coupons = {"SAVE10": 10, "HALF": 50}
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")
```

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

```
"""
Тесты для модуля factorial (Задание 2)
"""

import pytest
from factorial import factorial, factorial_recursive

# ========== Тесты для итеративной версии ==========


class TestFactorialIterative:
    """Тесты для итеративной версии факториала"""

    def test_factorial_zero(self):
        """Проверка: 0! = 1"""
        assert factorial(0) == 1

    def test_factorial_one(self):
        """Проверка: 1! = 1"""
        assert factorial(1) == 1

    def test_factorial_two(self):
        """Проверка: 2! = 2"""
        assert factorial(2) == 2

    def test_factorial_three(self):
        """Проверка: 3! = 6"""
        assert factorial(3) == 6

    def test_factorial_five(self):
```

```
    """Проверка: 5! = 120"""
    assert factorial(5) == 120

def test_factorial_ten(self):
    """Проверка: 10! = 3628800"""
    assert factorial(10) == 3628800

# Параметризованный тест для нескольких значений
@pytest.mark.parametrize(
    "n,expected",
    [
        (0, 1),
        (1, 1),
        (2, 2),
        (3, 6),
        (4, 24),
        (5, 120),
        (6, 720),
        (7, 5040),
    ],
)
def test_factorial_parametrized(self, n, expected):
    """Параметризованная проверка факториала"""
    assert factorial(n) == expected

# Тесты на исключения
def test_factorial_negative(self):
    """Проверка выброса ValueError для отрицательных чисел"""
    with pytest.raises(
        ValueError, match="Factorial not defined for negative numbers"
    ):
        factorial(-1)

def test_factorial_negative_large(self):
    """Проверка для большого отрицательного числа"""
    with pytest.raises(ValueError):
        factorial(-100)

def test_factorial_float(self):
    """Проверка выброса TypeError для чисел с плавающей точкой"""
    with pytest.raises(TypeError, match="n must be an integer"):
        factorial(5.5)

def test_factorial_string(self):
    """Проверка выброса TypeError для строк"""
```

```
        with pytest.raises(TypeError):
            factorial("5")

    def test_factorial_none(self):
        """Проверка выброса TypeError для None"""
        with pytest.raises(TypeError):
            factorial(None)

    # Граничные случаи
    def test_factorial_large_number(self):
        """Проверка для большого числа (должно работать без переполнения в Python)"""
        # Python поддерживает длинную арифметику
        result = factorial(20)
        assert result == 2432902008176640000

    def test_factorial_type_error_message(self):
        """Проверка сообщения об ошибке типа"""
        with pytest.raises(TypeError) as exc_info:
            factorial(3.14)
        assert "must be an integer" in str(exc_info.value)


# ========== Тесты для рекурсивной версии ==========


class TestFactorialRecursive:
    """Тесты для рекурсивной версии факториала"""

    @pytest.mark.parametrize(
        "n,expected", [(0, 1), (1, 1), (2, 2), (3, 6), (4, 24), (5, 120)]
    )
    def test_recursive_parametrized(self, n, expected):
        """Параметризованная проверка рекурсивной версии"""
        assert factorial_recursive(n) == expected

    def test_recursive_negative(self):
        """Проверка исключения для отрицательных чисел"""
        with pytest.raises(ValueError):
            factorial_recursive(-5)

    def test_recursive_type_error(self):
        """Проверка исключения для неверного типа"""
        with pytest.raises(TypeError):
            factorial_recursive(5.5)
```

```
    def test_recursive_large(self):
        """Проверка для большого числа"""
        assert factorial_recursive(10) == 3628800
        assert factorial_recursive(15) == 1307674368000


# ========== Сравнительные тесты ==========


def test_both_versions_agree():
    """Проверка, что обе версии дают одинаковые результаты"""
    for n in range(10):
        assert factorial(n) == factorial_recursive(n)
```

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

1) Реализуйте и протестируйте метод String repeat(String pattern, int repeat), который строит строку из указанного паттерна, повторённого заданное количество раз.

Спецификация метода:

repeat ("e", 0) = ""

repeat ("e", 3) = " eee "

repeat (" ABC ", 2) = " ABCABC "

repeat ("e", -2) = ValueError

repeat (None , 1) = TypeError

```
"""
Тесты для модуля string_repeat.

Содержит тесты для функций повторения строк.
"""


import pytest
from string_repeat import repeat_string


# ========== Параметризованные тесты ==========

@pytest.mark.parametrize("pattern,repeat_count,expected", [
    # Тривиальные случаи
    ("", 5, ""),
    ("e", 0, ""),
    ("e", 1, "e"),
    # Обычные случаи
```

```
    ("e", 3, "eee"),
    ("ABC", 2, "ABCABC"),
    ("123", 3, "123123123"),
    # С пробелами и спецсимволами
    (" ABC ", 2, " ABC  ABC "),
    ("\n", 3, "\n\n\n"),
    # Unicode
    ("Привет", 2, "ПриветПривет"),
    ("😀", 3, "😀😀😀"),
])
def test_repeat_valid_cases(pattern, repeat_count, expected):
    """Проверка корректных случаев повторения."""
    assert repeat_string(pattern, repeat_count) == expected


@pytest.mark.parametrize("pattern,repeat_count", [
    ("e", -1),
    ("test", -5),
    ("abc", -100),
])
def test_repeat_negative_count(pattern, repeat_count):
    """Отрицательное количество повторений -> ValueError."""
    with pytest.raises(ValueError, match="repeat count cannot be negative"):
        repeat_string(pattern, repeat_count)


@pytest.mark.parametrize("pattern", [None, 123, 3.14, [], {}, set()])
def test_repeat_invalid_pattern(pattern):
    """Неверный тип pattern -> TypeError."""
    with pytest.raises(TypeError):
        repeat_string(pattern, 1)  # type: ignore


def test_repeat_large_number():
    """Повторение большое количество раз."""
    result = repeat_string("a", 10000)
    assert len(result) == 10000
    assert result == "a" * 10000


def test_repeat_long_pattern():
    """Повторение длинного паттерна."""
    long_pattern = "abcdefghij" * 100  # 1000 символов
    result = repeat_string(long_pattern, 10)
    assert len(result) == 10000
```

```
    assert result[:1000] == long_pattern


def test_repeat_identity_property():
    """Свойство: repeat(x, 1) == x."""
    test_strings = ["a", "abc", "Hello", " ", "\n", "123"]
    for s in test_strings:
        assert repeat_string(s, 1) == s


def test_repeat_concatenation_property():
    """Свойство: repeat(x, a+b) == repeat(x, a) + repeat(x, b)."""
    pattern = "abc"
    a, b = 3, 4
    assert repeat_string(pattern, a + b) == \
            repeat_string(pattern, a) + repeat_string(pattern, b)


def test_repeat_length_property():
    """Свойство: len(repeat(s, n)) == len(s) * n."""
    test_cases = [
        ("a", 5),
        ("ab", 10),
        ("Hello", 7),
        ("", 100),
    ]
    for pattern, n in test_cases:
        assert len(repeat_string(pattern, n)) == len(pattern) * n
```

Вывод: В результате выполнения лабораторной работы были закреплены знания объектно-ориентированного программирования Python при решении задач.